

Meeting Splitter

Technical report and design rationale

Team Banana_Shake: Anay Gupta and Ajay Meena

Live application: https://choco-princess.github.io/meeting_spitter/

Public repository: https://github.com/Choco-princess/meeting_spitter

Purpose and engineering priorities

Meeting Splitter turns English meeting recordings into a clear, useful record: timestamped transcripts, refined terminology, organized minutes, decisions, action items and open questions. It helps participants revisit the discussion and understand what was agreed, what remains unresolved and what needs to happen next. The design prioritizes correctness, readable documentation and an accessible workflow.

The application combines a lightweight web interface with separate transcription, refinement and documentation stages. Users can inspect the original transcript, review corrections and trace generated notes back to the recording. Shared credentials stay on the backend, so the demo requires no API key from its users. This report explains the architecture, engineering decisions, verification and practical limitations.

What is implemented

- Three visible processing stages, with an optional additional review in the minutes stage.
- Four minutes-detail options and regeneration that reuses completed transcripts.
- Opt-in first-three-minute WAV/MP3 processing, including oversized originals.
- Source playback, visible refinement patches, nullable task fields and scope-labeled exports.

Thank you to IITG Techboard for the problem statement and the opportunity to build this project.

1. Architecture and request flow

Deployment topology

Layer	Responsibility
GitHub Pages / browser	React interface, local excerpt preparation, progress and exports
Cloudflare Pages gateway	Public API hostname; forwards to the Worker service binding
Cloudflare Worker	Input validation, fixed models/prompts, secret and provider handling
Groq inference	Whisper transcription and structured language-model output

Audio -> Whisper segments -> terminology proposals -> validated patches -> document draft -> optional targeted review -> browser playback and exports. The browser orchestrates separate requests so intermediate progress survives a later-stage failure in the open page.

Why this architecture

A static frontend is simple to publish, inexpensive and suitable for a small judging audience. The Worker provides a narrow server boundary for credentials and validation without running a persistent server. The Pages gateway is used because it provides the reachable API hostname in the deployed environment. Inference is delegated to Groq; no heavyweight models are downloaded into the browser.

Routing inference through a personal laptop would require that machine and network to stay available. Hardcoding a shared key into JavaScript would expose it to every visitor. The Worker secret avoids that exposure while keeping the demo convenient. This does not make public shared quota immune to abuse.

Why there is no database

Persistent accounts, recordings and session storage would add implementation and privacy work without being necessary for the initial submission. Current state lives in memory and exports provide a portable result. Reloading loses the run; persistent sessions are future scope.

2. Model selection and processing stages

Speech recognition: [whisper-large-v3](#)

Whisper handles English audio and returns timestamped segments suitable for source navigation. It is the trusted transcript baseline for the requested postprocessing evaluation, but that evaluation convention does not mean recognition is perfect. Names, jargon and unlabeled speakers can still be wrong. Automatic speaker diarization is not implemented.

Terminology refinement: [openai/gpt-oss-20b](#)

A separate smaller language stage proposes contextual corrections to names or technical terms. The optional glossary supplies spelling hints, not permission to invent content. The application applies short exact-span patches only when their original text matches. Ambiguous, overlapping or mismatched proposals are rejected. The raw transcript and change log remain visible.

This constrained edit approach reduces unnecessary rewriting and preserves the original evidence. It is deliberately less ambitious than polishing every sentence. Corrections can still be semantically mistaken, so refinement is not treated as verified ground truth.

Documentation: [openai/gpt-oss-120b](#)

The larger model is used for the more context-sensitive task: organizing discussion, detecting agreement, distinguishing proposals from decisions and identifying assignments. Structured output carries summary, topic notes, decisions, tasks, open questions and segment references. Missing owners or deadlines remain null rather than being guessed.

Free fallback: [qwen/qwen3.8-27b](#)

A distinct free model is used when the primary documentation model has exhausted its daily allowance. The fallback is conditional, and results report actual generation/review model IDs. This retains a usable free demo but introduces model-dependent variation. The application never silently claims that a fallback result came from the primary model.

These choices separate recognition, narrow correction and documentation responsibilities. Browser-hosted inference was deferred because model download size, hardware support, performance and recognition quality would introduce uncertainty under the submission deadline. No paid fallback is configured.

3. Correctness and corrective review

Grounding and classification

Documentation is instructed to stay within the supplied transcript. It may paraphrase, but should not promote a suggestion into an agreement, a question into an assignment, or an historical policy into a new decision. Nearby names do not reliably identify an unlabeled speaker. Explicit assignments or later named recaps can support ownership; otherwise ownership stays unspecified.

Prompts address cases encountered during development: earlier passed votes followed by a separate pending motion, fragmented questions followed by later answers, relative-date referents and ambiguous terms such as display boards. A supported public statement is described as a statement or reported concern, rather than automatically becoming an established fact.

Optional extra accuracy review

Review is enabled by default in the interface. It receives the transcript and candidate document and checks factual support, coverage and commitment classification. It returns nullable replacement fields: null keeps the corresponding sound draft field, while a populated field replaces that field completely. This avoids unnecessary full-document rewriting and preserves useful candidate material.

The extra pass costs more quota and latency. It can repair mistakes but can also miss or introduce errors. It is an additional model check, not an independent factual authority, formal proof or a guarantee of zero hallucinations. If review fails, the candidate draft remains available; the user can retry or explicitly disable review and reuse the draft.

Structural safeguards and source references

Schemas enforce valid output shapes and permitted categories. Reference cleanup retains valid source segments, and timestamp controls lead back to original playback. Deterministic rules check patch safety and normalize unresolved classifications. These safeguards reduce malformed or unsupported structure, but they cannot prove that a paraphrase is entailed by the source.

Balanced usefulness

The system allows readable synthesis instead of requiring every sentence to be a literal quote. Detail levels change desired coverage without forcing word counts or invented padding. Important statements should remain reviewable, and uncertainty should be visible. Users must still check consequential names, dates, numbers and assignments against the recording.

4. Audio limits, detail levels and user workflow

First-three-minute excerpt mode

The full-upload cap remains 24 MiB. For a larger standard WAV or MPEG Layer III MP3, the user can opt into an excerpt. The browser reads a bounded prefix, retains complete frames, decodes locally and resamples to mono 16 kHz PCM WAV. At 180 seconds, the normalized mono PCM payload is approximately 5.76 MB, allowing an oversized original to produce a small upload.

The original file stays unchanged and remains the playback source. Scope notices identify the first three minutes in the interface and all relevant exports. This is a deliberate partial analysis, not a summary of the entire recording. Unfinalized headers, cut frames, unusual WAV encodings and unsupported oversized formats require a valid shorter export.

Four minutes-detail tiers

Quick recap requests the essential overview; Standard requests ordinary useful minutes; Detailed is the default for fuller topic coverage; Full notes requests the most descriptive account. Changing the level reuses raw and refined transcripts and regenerates only documentation. Under restrictive free output caps, the desired distinction can be compressed; a warning explains this limitation.

Failure recovery and cancellation

The UI preserves completed raw/refined stages and candidate documentation during later failures. Cancel marks the run as paused and guards against stale responses updating a replaced file. A retry resumes useful work rather than restarting transcription. Provider work already accepted may still finish after cancellation. Reloading discards in-memory progress.

Exports and privacy

TXT exposes raw/refined transcripts; Markdown presents readable minutes; JSON retains structured fields and model metadata; ZIP bundles results. The app does not persist audio or transcripts on its backend, but both pass through Cloudflare to Groq and are subject to those services' policies. Local excerpt preparation reduces the amount uploaded when selected.

5. Free-service constraints and verification

Quota failure diagnosis

Testing exhausted the primary documentation model's daily allowance. The fallback also exposed a separate output-tokens-per-minute cap: some requested output reservations could never fit, even after waiting. Treating that condition as an ordinary temporary busy response caused ineffective repeated waits.

The final backend recognizes restrictive output limits, reduces the reservation and requests concise structured output. For the most restrictive Qwen response, reasoning effort is disabled to preserve output capacity. The transcript is not truncated to fit this adjustment. A visible warning explains that the result may be more compact than the selected level. Temporary rate limits still use provider retry information.

A daily allowance can replenish; that does not remove a per-minute cap. Exact reset time is not assumed to be midnight in India. Free access remains dependent on the account's available quota, and overlapping visitors may still encounter delays or failures. No unlimited-availability claim is made.

Completed validation

- 41 deterministic tests passed; TypeScript checking and the production build passed.
- Nine live semantic cases passed at prompt version 1.9, immediately before the quota-handling change.
- The fresh randomly selected YouTube holdout completed on the public app at version 1.10, including review and restrictive output handling. Separate deployment quantities were retained; confirmed decisions and tasks were correctly empty.
- All four completed MP3s in tests_anay passed strict full-file decoding and first-three-minute transcription. Two unfinished/corrupt WAVs were skipped.
- A valid 55 MiB WAV excerpt completed the browser pipeline and scope-labeled exports. A 157 MiB MP3 completed preparation, transcription and refinement in an earlier run, then hit quota.

Browser input errors, cancellation and a 390 px responsive layout were also checked. A ten-minute recording produced Quick recap, but the full four-level live comparison was interrupted by quota/provider errors. The nine-case suite is not claimed to have been rerun at version 1.10. Detailed records remain local outside the public repository.

6. Evaluation method, tradeoffs and future work

Independent postprocessing reference

For real recordings, Whisper text was inspected first and an independent expected-content reference was written before looking at generated notes. Comparison focused on facts, unanswered questions, pending versus passed motions, ownership, deadlines, numbers and omissions. This follows the requested convention of trusting Whisper for postprocessing assessment; it is not an independent audit of speech recognition accuracy.

The new holdout was selected once from a previously downloaded pool after a code checkpoint. Its expected content was recorded before examining model output. The result correctly described an existing Kubernetes deployment tool rather than inventing new work, preserved distinct deployment counts, and did not claim a demonstration succeeded beyond the transcript's scope.

Observed tradeoffs

Incremental development established a functioning submission before expanding controls. Narrow refinement preserves auditability but may leave transcription problems unresolved. A second model pass can improve correctness but consumes scarce free quota. Excerpts keep large files usable but omit later discussion. Structured schemas improve reliability of the interface but cannot certify meaning. Free fallback improves availability while reducing consistency and sometimes detail.

Earlier real-recording rounds exposed overbroad claims that questions were unanswered, uncertainty in speaker attribution and ambiguity around boards. Prompt changes targeted these cases. The latest folder round completed three of four reviewed results; one review was blocked before the final output-cap fix. Remaining attribution and coverage weaknesses must not be interpreted as solved globally.

Future scope, in priority order

- Expand independent real-meeting evaluation and complete the same-transcript four-tier comparison under available quota.
- Add careful chunking and cross-chunk merging for longer transcripts, retaining source identity and detecting repeated or conflicting commitments.
- Import speaker-labeled meeting transcripts; add diarization only after measuring quality and cost.
- Improve quota status, user-supplied key support and abuse protection.
- Add editable minutes and persistent sessions with an explicit retention policy.
- Investigate local inference and browser models once device support and quality can be validated.

References and implementation pointers

Primary implementation: `src/App.tsx`, `src/api.ts`, `src/excerpt.ts`, `shared/schema.ts`, `worker/index.ts` and `worker/prompts.ts` in the linked public repository. Deployment configuration: `gateway/`, `wrangler.jsonc` and `.github/workflows/pages.yml`.

Groq rate-limit documentation: <https://console.groq.com/docs/rate-limits> . This report describes the implemented account behavior; it does not promise future provider quotas or prices.